Journal of Circuits, Systems, and Computers
Vol. 27, No. 6 (2018) 1850095 (11 pages)
© World Scientific Publishing Company
DOI: 10.1142/S0218126618500950

J CIRCUIT SYST COMP Downloaded from www.worldscientific.com
by UNIVERSITY OF GOTHENBURG on 10/19/17. For personal use only.

A Dynamic Equilibrium Algorithm to Enhance Randomness in TRNG*

Chenyang Guo† and Yujie Zhou‡

SEIEE, Shanghai Jiao Tong University,
No. 800 Dongchuan Road, Shanghai,
200240, P. R. China
†*guochenyang@sjtu.edu.cn*
‡*zhou863@vip.sina.com*

Received 15 January 2017
Accepted 11 September 2017
Published 11 October 2017

In this paper, a new method is proposed for randomness enhancement. The approach is called the dynamic equilibrium algorithm (DEA). It is used to solve the problems existing in the true random number generator (TRNG). First, the advantages and defects of LFSR as a post-processing module are discussed. When sampling 1000 groups of data, only 517 groups can pass all 15 tests in SP800-22 with a pass rate of 0.981. DEA is actually a great solution to this problem. The essence of DEA is to guarantee the approximately uniform distribution of the overlapping template to improve the bit-entropy by the compression of the data. This method is easy to implement in both software and hardware. The pass rate increases more than 40% with a low compression rate.

Keywords: Dynamic equilibrium algorithm (DEA); true random number generator (TRNG); linear feedback shift register (LFSR).

1. Introduction

Random number generators (RNGs) are widely used in security chips. Most of the cryptographic algorithms are supported by the random numbers that these algorithms generate. They are the key to the security of a chip. Thus, RNG has been the hotspot of information security research in recent years. An efficient RNG is required not only to generate binary random sequence, but also to be integrated at a small scale and with low power consumption.

Ideal random experiments are unpredictable, independent and unbiased. The goal of any RNG is clearly to generate random sequences from ideal random experiments. However, real-world RNGs can only achieve this goal approximately. So many statistical

*This paper was recommended by Regional Editor Emre Salman.
† Corresponding author.

tests, like SP800-22 and AIS 31, help to determine to what extent the sequence is random. To generate the sequence with more randomness, it is important to understand the nature of RNG.

Generally, an RNG consists of a nondeterministic part (entropy source) that generates unpredictable digital data, and a deterministic part that generates the output sequence of the RNG (random numbers) from this data. The non-deterministic part of the RNG exploits an entropy source, often physical, to generate a raw random number sequence, which is deterministically post-processed. Either the deterministic part or the nondeterministic part may be omitted, giving a pure physical true random number generator (PTRNG) or a pure deterministic RNG (DRNG), respectively.[1]

The core of a PTRNG is the entropy source. It takes advantage of the unpredictability of the noise in nature, for example, the thermal noise in the circuit, the random jitter of the clock, etc.[2–6] Recently, researchers also used photons, to achieve a random number sequence. However, these are difficult to implement on chips because of the high cost on both price and area. The most widely used random source on a chip is the oscillator-based random source.

SP800-22 and AIS 31 specify the uniform evaluation criteria for an RNG. In the entropy source, the possibility of the next bit being 1 or 0 is hoped to be 50% or close to 50%. In addition, when observing a long sequence from the entropy source, we hope that each overlapped or nonoverlapped template has equal probability, which means greater bit-entropy. Unfortunately, the bit sequence generated by the entropy source may not have good statistical properties due to the interference of the environment. In fact, it is almost impossible to accurately and directly calculate the probability of the next bit from the entropy source because of the complex effects of temperature, atmospheric pressure and other environmental factors. Worse still, researchers found that in different environments, different designs and different placements of components, or even in two different experiments under the same conditions, the pass rate is not the same. Because it is difficult to change the entropy source directly, post-processing is needed.

A DRNG, also known as a pseudo random number generator (PRNG), may be viewed as a finite automaton that receives input (a seed and possibly also additional external input). The output from PRNG usually has good statistical characteristics. Some PRNGs, such as linear feedback shift registers (LFSRs) are easy to implement in hardware. However, the current output of a PRNG depends on the current internal state and additional input according to a mathematical algorithm. In addition, the sequence from a PRNG also has periodicity. In Li's paper,[7,8] he uses reseeding method to extend the period of the chaotic map. The period length is greater than 2^{253}. In Bemdt's work,[9] he proposes a hardware oriented 80-bit-key binary additive stream cipher. And the period is extended to greater than 2^{249}. Although the period can be extended to a large integer in PRNG, it still has the possibility to be detected by attackers, and the cryptographic system would be threatened. Therefore, a PRNG is only applied in systems with a low security threshold.

J CIRCUIT SYST COMP Downloaded from www.worldscientific.com
by UNIVERSITY OF GOTHENBURG on 10/19/17. For personal use only.

J CIRCUIT SYST COMP Downloaded from www.worldscientific.com
by UNIVERSITY OF GOTHENBURG on 10/19/17. For personal use only.

LFSRs are popular PRNGs because of the low area cost and high speed of throughput.[10] If the feedback polynomial is a primitive polynomial of degree m, we call the output bit sequence an m-sequence. Depending on the property of the m-sequence, the output bit sequence has good statistical properties and it easily passes the Frequency Test, Runs Test, Test for the Longest Run of Ones in a Block, Runs Test and Serial Test in SP800-22. However, it fails the Binary Matrix Rank Test, Discrete Fourier Transform Test and Linear Complexity Test. This is because an m-sequence has a linear property and has a period of 2^{m-1}. By observing 2^m consecutive bits, we can easily calculate the primitive polynomial.[11] This endangers the security chips, so we cannot generate random numbers using an LFSR by itself.

A common improvement to this method is that in every clock the feedback bit from an LFSR is XORed with the bit from a true random source so that the linear property is eliminated. At the same time, due to the unpredictability of the bit sequence from the random source, this method has destroyed the original statistical characteristics of the LFSR. Although it still passes many tests with a high pass rate, the Serial Test remains too low.

The focus of the Serial Test is the frequency of all possible overlapping m-bit patterns across the entire sequence. The purpose of this test is to determine whether the number of occurrences of the 2^m m-bit overlapping patterns is approximately the same as would be expected for a random sequence. We require random sequences to have uniformity; that is, every m-bit pattern has the same chance of appearing as every other m-bit pattern.[12]

In this brief, we propose a new compression algorithm, for generating random data. This method aims to improve the pass rate of the Serial Test while keeping other tests with a high pass rate. In the process of sampling data, we achieve the bits meeting our conditions and discard the biased data. Then, the bit sequence generated can pass the Serial Test with a high pass rate.

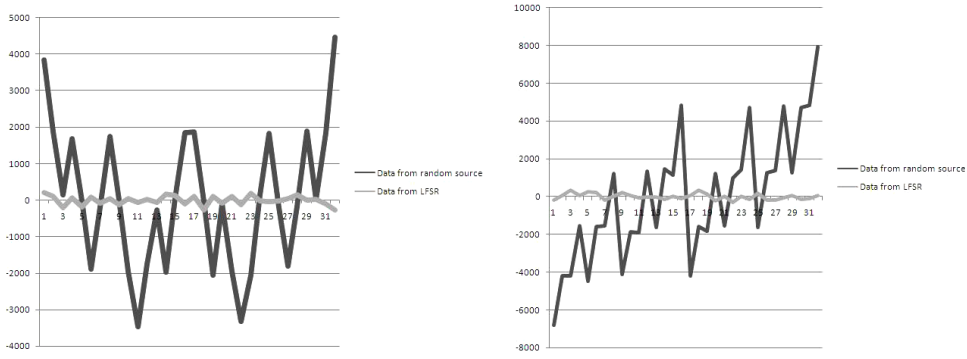
2. Structure of RNG

2.1. *Random source*

A random source is the core of the whole random number generator. It is a hot research topic in the field of information security in recent years. Three techniques are often presented in papers: direct amplification of a noise source,[2] oscillator-based random sources,[3–5] and discrete-time chaotic maps.[6,10]

However, in the process of producing data, the statistical properties of the random sequence are influenced by environmental factors.

Bit-entropy, which is based on the theory by Shannon in 1948,[13] is a classic method to measure the randomness of a source. In our work, we choose the frequency of a 5-bit overlapping template as an alternative. It is an intermediate result of the bit-entropy algorithm and it can be directly reflected in the figure.



(a) (b)

Fig. 1. (a) Random source suffers by insufficient jitter. (b) Random source suffers by deviation of duty-cycle.

Obtain 1 million consecutive bits from an oscillator-based random source. Due to the insufficient jitter of low frequency clocks and the large deviation of duty-cycle caused by the environment, the randomness of the sequence deteriorates. The dark line in Fig. 1 shows the effect of the two environmental factors on such a random sequence.

The horizontal coordinate is the 5-bit overlapping template. For example, 0 means 00000 and 31 means 11111. The vertical coordinate is the difference of templates between the test value and the standard value. In our work, 1,000,000 bits are sampled and the standard value is $1{,}000{,}000/32 = 31{,}250$.

From Fig. 1, we know that insufficient jitter causes the long runs of '0's and '1's and that deviation of duty-cycle causes the biased frequency distribution. These all contribute to the lack of randomness and result in a low passing rate in SP800-22. The test result is shown in the first two columns in Table 1. Most values are less than the standard value 0.981. So a post-processing module is necessary.

2.2. *Linear feedback shift register (LFSR)*

A LFSR is actually a pseudo-random generator. Due to the linear characteristics and periodicity, it cannot be used as a random number generator alone. However, with small area, high output rate and good statistics, it is often applied as a post-processing module in a TRNG. The output bit from a random source XORed with the feedback of the LFSR constructs the classical random number generator in Fig. 2.

In our work, a degree 31 LFSR is used with the primitive polynomial:

$$f(x) = x^{31} + x^{27} + x^{23} + x^{19} + x^{15} + x^{11} + x^{10} + x^{9} + x^{7} + x^{6} + x^{5} + x^{3} + x^{2} + x + 1. \quad (1)$$

The shallow line in Fig. 1 represents the number of 5-bit overlapping templates. After post-processing by the LFSR, the number of overlapping templates stays close to the standard value.

J CIRCUIT SYST COMP Downloaded from www.worldscientific.com
by UNIVERSITY OF GOTHENBURG on 10/19/17. For personal use only.

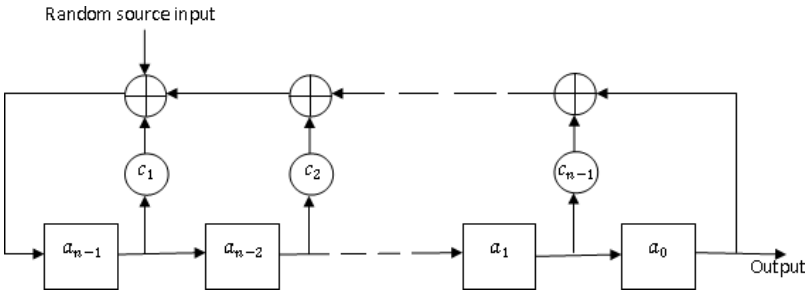


Fig. 2. Structure of LFSR.

J CIRCUIT SYST COMP Downloaded from www.worldscientific.com
by UNIVERSITY OF GOTHENBURG on 10/19/17. For personal use only.

In fact, if the feedback polynomial of LFSR is a primitive polynomial, the pseudo random sequence has a maximum period of 2^{m-1}. If the state of the m registers is combined into an integer between 1 and 2^{m-1}, then each integer appears in this period once. When sampling continuous 2^{m-1} bits, the pseudo random sequence has uniform frequency distribution, large entropy and other good statistical properties. However, after the output bit from the random source is XORed with the feedback bit, during the same period, some integer doesn't appear and some integers appear multiple times. The output sequence from the LFSR no longer processes the good statistical properties. Although the linear and periodic properties are broken by the random source, the statistical properties of the LFSR are also destroyed by the same random source.

This problem results in the low pass rate of 15 tests in SP800-22. After sampling 1,000 groups of 1000*1,000,000 bits, only 517 of 1,000 groups pass all 15 tests with a

Table 1. Test results of random source and LFSR.

Tests**	Random Source 1	Random Source 2	LFSR only	TRNG
Frequency	0.983	0.000	0.999	0.993
Block Frequency	0.023	0.000	0.999	0.988
Cumulative Sums*	0.977	0.000	0.999	0.985
Runs	0.010	0.000	0.998	0.994
Longest Run	0.717	0.000	0.999	0.986
Rank	0.993	0.989	0.000	0.987
FFT	0.972	0.000	0.000	0.982
Non Overlapping Template*	0.028	0.000	0.989	0.978
Overlappling Template	0.016	0.000	0.995	0.987
Universal	0.887	0.000	0.999	0.986
Approximate Entropy	0.003	0.000	0.999	0.987
Random Excursions*	0.864	0.000	0.984	0.983
Random Excursions Variant*	0.969	1.000	0.987	0.989
Serial Test* ($m = 16$)	0.887	0.000	0.999	0.986
Linear Complexity	0.986	0.992	0.000	0.987

Notes: *This test consists of several sub-tests: the average result is reported. **Random Source 1 represents the left image in Fig.1. Random Source 2 represents the right image in Fig. 1. TRNG represents the data from Random Source 1 post-processing by LFSR.

pass rate of 0.981 and 476 of the rest of the groups fail in the Serial Test. Thus, a correction algorithm is needed and is discussed in the following section.

3. Dynamic Equilibrium Algorithm (DEA)

In this section, the new algorithm called the dynamic equilibrium algorithm (DEA) is proposed.

3.1. *Steps for DEA*

Let v_0, v_1, v_2, v_3 denote the frequency of the pattern 00,01,10,11 in the circularized string of N bits. And define $D[0], D[1], D[2], D[3]$ as the deviation by (2)

$$D[j] = v_j - \frac{N}{4}, \quad j = 0, 1, 2, 3. \tag{2}$$

N is the total number of the bits. For example, to a random string 0100011001011101, we copy the first bit 0 to the end of the string to make the string circularized. Then N is 16, v_0 is 3, v_1 is 5, v_2 is 5, v_3 is 3. And $D[0], D[1], D[2], D[3]$ is −1,1,1,−1.

Let $D_{\max} = \max\{|D[j]|\}, j = 0, 1, 2, 3$. So $D_{\max}$ is 1 in the last example.

Ideally, $D[j]$ is equal to 0 as a uniform distribution. Then, we calculate the *sum* of the $D[j]$ in (3):

$$\text{sum} = \sum_{j=0}^{3} |D[j]|. \tag{3}$$

Obviously, *sum* must be less enough to pass the statistical test like SP800-22 or AIS31. We will use a statistical method to determine the threshold that the sum should be less than. However, the threshold is approximate and we will illustrate it in the following parts.

The steps of DEA are as follows:

(i) Obtain 2 K random Bytes from TRNG. Calculate the $D[j]$ and *sum* by (3). $N = 2\,KB$.
(ii) Take another 128 bits from TRNG. Update the value of $D[j]$ and *sum*. If *sum* is less than $T(N)$ or $D_{\max}$ is reduced, the 128 bits are retained, $N = N + 128$. If not, these 128 bits are abandoned.
(iii) If $N = 4\,KB$, do 1, else do 2.

For example, if in the origin 2 K random Bytes, $D[0] = -55, D[1] = -47, D[2] = -47, D[3] = 149$, then the *sum* is 298. $D_{\max}$ is 149.

Take another 128 bits from TRNG. After updating, $D[0] = -44, D[1] = -51, D[2] = -51, D[3] = 146$, the *sum* is updated to 292 and the threshold $T(N)$ is 256, then *sum* is larger than $T(N)$. But $D_{\max}$ is reduced from 149 to 146. Then the 128 bits are retained.

J CIRCUIT SYST COMP Downloaded from www.worldscientific.com
by UNIVERSITY OF GOTHENBURG on 10/19/17. For personal use only.

This is known as DEA. With DEA, 1 million bits are obtained. Then, all the registers are cleared, and another 2 KB data are read. Repeat the steps above to get another 1 million bits.

3.2. *Proposed threshold value*

$T(N)$ is the threshold of DEA and it is the key to this method. It will be introduced in detail in this section. A good threshold will guarantee a high pass rate and a low compression rate at the same time. Statistical methods are used to determine the threshold.

Actually, a *sum* greater than $T(N)$ does not mean failure in the Serial Test because it is not a necessary and sufficient condition. We need to calculate the pass rate of each sum. In theory, 4 KB of data has a total of 2^{32768} samples. All samples should be calculated to determine the threshold. However, this approach is not realistic. First, the amount is too large. Secondly, for a certain TRNG, due to the influence of environmental factors, the possibility of appearance of each template is not equal. Consequently, the method of sampling is used.

First, we get 348 M random bytes from TRNG with LFSR, a total of 98,304 sets of sum values, and the sum values of 4 K random bytes which can pass the Serial Test is from 0 to 614 while the sum values failed is from 468 to 1296. Thus, we are mainly concerned about the sum values from 468 to 614. There are 13,263 sets of data are in this range, and the pass rate of data in this range is illustrated in Fig. 3.

We see that when the sum value is in the range of 468 to 564, the pass rate stays at a high level. When the sum value is greater than 564, the pass rate has a significant

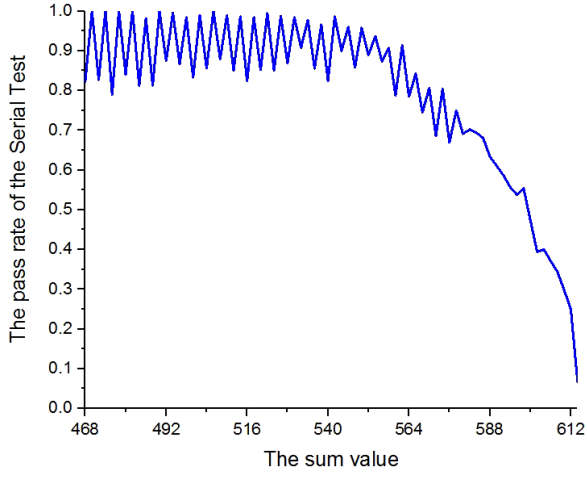


Fig. 3. Pass rate of sum value from 468 to 614. X-axis is sum value and Y-axis is pass rate.

J CIRCUIT SYST COMP Downloaded from www.worldscientific.com
by UNIVERSITY OF GOTHENBURG on 10/19/17. For personal use only.

downward trend that is approximately linear. As a result, the threshold of 4 KB is set to 564.

3.3. *Compression rate of the random data*

In addition to the pass rate of the data, the compression rate is another critical point in this method. Clearly, the statistical property of input data significantly influences the compression rate. So DEA cannot be used alone as a post-processing method, but it can be used as the third part of TRNG (after random source and LFSR). For the problems that LFSR cannot solve, DEA gives a good optimization.

The value of the compression rate depends on the design of the RNG, the environment and the threshold in the DEA. Designers of the RNG have to adjust the threshold to obtain a low compression rate. In our work, we sample 1,000 groups of 128 M random bytes with a compression rate between 4.8% and 9.1%.

3.4. *For the security of the random data*

The random data generated by DEA can be regarded as part of the original random data. This method does not affect the security of the original data. In 16 bytes of data, there are 2^{128} combinations. With DEA, even if attackers know the value of the counter and the threshold, the numbers of 16 Byte combinations may decrease only to 2^{127} due to Step 2.

4. Results

In this section, we will compare DEA to other post-processing algorithms with respect to statistical properties and robustness.

NIST SP800-22 is used to evaluate the randomness of our system. In the test, 1,000 sequences will be generated by DEA, and each sequence will have the length of 1 million bits. The P-value in each test is fixed to 0.01 to pass the test. As suggested in SP800-22, the minimum pass rate is 0.9806, that is, at least 981 groups out of 1,000 groups must pass the test.

4.1. *Serial test enhancement with DEA*

First, we conduct an experiment to prove the enhancement against the Serial Test with DEA. We take six groups of data without DEA for the Serial Test. Then, we take the same six groups with DEA. The pass rate is shown in Table 2. In last two rows, with DEA the pass rate increases obviously.

4.2. *Comparison with previous work*

In this section, we will compare our work with both the TRNGs and the PRNGs in previous work.

J CIRCUIT SYST COMP Downloaded from www.worldscientific.com
by UNIVERSITY OF GOTHENBURG on 10/19/17. For personal use only.

Table 2. Test results with or without DEA. This test consists of several sub-tests: the worst result is reported.

Serial test ($m = 2$) without DEA	Serial test ($m = 2$) with DEA	Serial test ($m = 5$) without DEA	Serial test ($m = 5$) with DEA
0.977	0.991	0.985	0.993
0.970	0.986	0.987	0.995
0.985	0.995	0.979	0.989
0.987	0.994	0.975	0.986
0.989	0.996	0.988	0.994
0.984	0.991	0.983	0.992

J CIRCUIT SYST COMP Downloaded from www.worldscientific.com
by UNIVERSITY OF GOTHENBURG on 10/19/17. For personal use only.

Compared with the PRNG, one advantage of the TRNG is that its period is theoretically infinite while the period of the PRNG is finite. Even the period is usually a very large integer. That means the PRNG is suitable in some fields where the security level is not considered high enough for example circuit test, the gambling and so on. In some fields, which need high security level, like financial IC card or some cryptography applications, PRNG is not suitable. So we preferred TRNG for these fields.

In Table 3, the test results for DEA are compared with those for the DMLM-PRNG[14] and for the chaotic-map[15] and for the RM-PRNG.[9]

It can be seen from the table that the other 14 tests of SP800-22 are not significantly changed with DEA.

Table 3. Comparison with other RNGs.

Tests	Without DEA	With DEA	DMLM-PRNG[14]	Chaotic-map[15]	RM PRNG[9]
Frequency	0.982	0.996	0.9917	0.990	0.985
Block Frequency	0.991	0.990	0.9750	0.975	0.985
Cumulative Sums*	0.978	0.995	0.9917	0.985	0.985
Runs	0.986	0.993	0.9917	0.995	0.995
Longest Run	0.988	0.991	0.9917	0.990	0.99
Rank	0.989	0.991	0.9833	0.985	0.99
FFT	0.991	0.990	0.9750	0.990	0.995
NonOverlapping Template*	0.987	0.988	0.9816	0.970	0.97
Overlappling Template	0.991	0.991	0.9917	0.985	0.99
Universal	0.991	0.986	0.9833	0.980	0.99
Approximate Entropy	0.990	0.997	0.9833	0.990	1
Random Excursions*	0.988	0.995	0.9945	0.9796	0.9855
Random Excursions Variant*	0.987	0.998	0.9983	0.9864	0.9855
Serial Test* ($m = 16$)	0.976	0.995	0.9833	1	0.985
Linear Complexity	0.992	0.996	0.9833	0.990	0.995

Note: *This test consists of several sub-tests: the average result is reported.

4.3. *Solutions to the previous problem in classic TRNG*

To compare with the original TRNG (random source and LFSR), 1,000 groups of 1,000*1,000,000 bits are sampled from TRNG without DEA. The results show that only 517 groups of random data pass all of the 15 tests in SP800-22. When the same data are processed by DEA, there remain 931 groups of 1,000*1,000,000 bits due to the compression of the data. Of these 931 groups, 928 pass all of the 15 tests. The pass rate increased from 51.7 % to 99.6 %. Thus, DEA enhances the randomness of the original TRNG.

5. Conclusions

In this brief, we first propose the problem of the original TRNG. Although an LFSR improves the statistical properties of an output sequence from a random source, its own output sequences good statistical properties are broken by the random source. By testing a large number of random sequences from the original TRNG, we see that the 15 tests in SP800-22 are not always passed. The main reason is that the Serial Test cannot reach the pass rate of 0.9806. So DEA is proposed to solve the problem. The experimental results show that the randomness of the sequence with DEA improves significantly, with a low compression rate between 4.8% and 9.1%. This algorithm can be implemented in both software and hardware easily.

Acknowledgments

The authors declare that there is no conflict of interest regarding the publication of this paper.

References

1. BSI: Application notes and interpretation of the scheme (AIS) 31 functionality classes and evaluation methodology for physical random number generators, Version 1 (25.09.2001), English translation.
2. W. T. Holman, J. A. Connelly and A. B. Downlatabadi, An integrated analog/digital random noise source, *IEEE Trans. Circuits Syst. I* **44** (1997) 521–528.
3. B. Jun and P. Kocher, The intel random number generator, *Cryptography Research Inc.*, white paper, 1999.
4. M. Dichtl and N. Janssen, A high quality physical random number generator, *Proc. Sophia Antipolis Forum Microelectronics* (SAME, 2000), pp. 48–53.
5. C. S. Petrie and J. A. Connelly, Modeling and simulation of oscillator-based random number generators, *Proc. IEEE Int. Symp. Circuits and Systems (ISCAS 96)*, Vol. 4, (1996), pp. 324–327.
6. T. Stojanovski and L. Kocarev, Chaos-based random number generators part I: Analysis, *IEEE Trans. Circuits and Syst. I* **48** (2001) 281–288.
7. C. Y. Li, T. Y. Chang and C. C. Huang, A nonlinear PRNG using digitized logistic map with self-reseeding method, *Int. Symp. Vlsi Design Automation and Test*, (IEEE, 2010), pp. 108–111.

J CIRCUIT SYST COMP Downloaded from www.worldscientific.com
by UNIVERSITY OF GOTHENBURG on 10/19/17. For personal use only.

8. C. Y. Li *et al.*, Period extension and randomness enhancement using high-throughput reseeding-mixing PRNG, *IEEE Trans. Very Large Scale Integr. Syst.* **20** (2012) 385–389.
9. B. M. Gammel, R. Gottfert and O. Kniffler, An NLFSR-based stream cipher, *IEEE Int. Symp. Circuits and Systems*, (IEEE, 2006), pp. 2920.
10. J. C. Cerda *et al.*, An efficient FPGA random number generator using LFSRs and cellular automata, *2012 IEEE 55th Int. Midwest Symp. Circuits and Systems (MWSCAS)*, (IEEE, 2012), pp. 912–915.
11. I. Pomeranz and S. M. Reddy, On methods to match a test pattern generator to a circuit-under-test, *IEEE Trans. Very Large Scale Integr. Syst.* **6** (1998) 432–444.
12. A. Rukhin, J. Soto, J. Nechvatal *et al.*, A statistical test suite for random and pseudo-random number generators for cryptographic applications, Booz-Allen and Hamilton Inc Mclean Va, 2001.
13. C. E. Shannon, A mathematical theory of communication, *Bell. Syst. Tech. J.* **27** (1948) 379–423.
14. S. L. Chen, T. T. Hwang and W. W. Lin, Randomness enhancement using digitalized modified logistic map, *IEEE Trans. Circuits Syst. II, Exp. Briefs*, **57** (2010) 996–1000.
15. T. Addabbo, M. Alioto, A. Fort *et al.*, A class of maximum-period nonlinear congruential generators derived from the Rnyi chaotic map, *IEEE Trans. Circuits Systems I, Regular Papers* **54** (2007) 816–828.

J CIRCUIT SYST COMP Downloaded from www.worldscientific.com
by UNIVERSITY OF GOTHENBURG on 10/19/17. For personal use only.